

Back Office API

Introduction

This document describes API exposed by *Back Office*.

The key words `MUST`, `MUST NOT`, `REQUIRED`, `SHALL`, `SHALL NOT`, `SHOULD`, `SHOULD NOT`, `RECOMMENDED`, `MAY`, and `OPTIONAL` in this document are to be interpreted as described in [RFC 2119](#).

GraphQL endpoint

GraphQL API is exposed under `/graphql` endpoint.

Introspection

GraphQL comes with fantastic [introspection](#) system which allows you to see the API.

The API schema is generated dynamically based on account permission so different account `MAY` see different schemas for different users.

There also GraphiQL web client available in the Back Office under `/backoffice/graphiql` to preview and test API in the browser.

Authorisation

The system use JWT authorisation with expiring keys. Mutation `login` generates new token.

Additionally, it checks time since last activity and requires re-authorisation after X hours (typically 4 hours) . It will return `SESSION_EXPIRED` error when the session expires.

```
mutation {  
  login(email: "my-account@example.com", password: "my-password") {  
    token  
  }  
}
```

For the following requests you need to add the following header: `Authorization: Bearer <TOKEN>`

Standard queries

There is standard format of query parameters however this is `OPTIONAL` so not all queries follow it.

```
query ($limit: Int, $offset: Int, $sort: Sort, $filter: [Filter], $options: JSONObject) {  
  gameWin(limit: $limit, sort: $sort, offset: $offset, filter: $filter, options: $options) {  
    meta {  
      total  
      limit  
      offset  
    }  
    items {  
      day  
      nativeId  
    }  
  }  
}
```

```
    currency
  }
}
```

Query variables:

```
{
  "limit": 2,
  "offset": 10,
  "sort": {
    "field": "day",
    "order": "DESC"
  },
  "filter": [
    {
      "type": "LIKE",
      "field": "nativeId",
      "value": "%test%"
    },
    {
      "type": "EQUAL",
      "field": "currency",
      "value": "eur"
    }
  ],
  "options": {
    "convert": true,
    "interval": "day",
    "dimensions": []
  }
}
```

Limit

Limits number of returned `items` .

Warning: There is hard limit of 100 000 `items` that can be fetched at once.

Offset

Skips selected number of returned `items` .

Sort

Sorts items by given `field` and `order` (`DESC` or `ASC`).

The system analyzes the cost of sorting and if detects too high cost it will reject the query with suggestion to use more filters to narrow down the data and make query less expensive.

Filters

Filters the column. There are following types of filters. Certain fields have only selected types of filters available.

Type	Description	Value type	Value example
<code>IN</code>	Checks if contains the value	<code>[String]</code>	<code>["a", "b"]</code>

Type	Description	Value type	Value example
EQUAL	Checks if values are equal	String or will be mapped to string	14
NOT_EQUAL	Checks if values are not equal	String or will be mapped to string	14
LIKE	Checks if value matches <code>value</code> . You can use <code>%</code> to substitute any string	String or will be mapped to string	%test%
GREATER	Checks if value is greater than the field value	[String]	"10"
GREATER_OR_EQUAL	Checks if value is greater than or equal to the field value	[String]	"10"
LOWER	Checks if value is lower than the field value	[String]	"10"
LOWER_OR_EQUAL	Checks if value is lower than or equal to the field value	[String]	"10"
NULL	Checks if value is null	N/A	N/A
NOT_NULL	Checks if value is not null	N/A	N/A

Options

Additional non-standard options.

Response

```
{
  "data": {
    "gameWin": {
      "meta": {
        "total": 52475,
        "limit": 2,
        "offset": 10
      },
      "items": [
        {
          "day": "2021-03-09",
          "nativeId": "test-native-id",
          "currency": "usd"
        },
        {
          "day": "2021-03-10",
          "nativeId": "test-native-id-2",
          "currency": "eur"
        }
      ]
    }
  }
}
```

Meta

- `total` - total number of `items`.
- `limit` - limit of returned `items`.
- `offset` - number of skipped `items`.

- `hasPrev` - if previous page is available
- `hasNext` - if next page is available

Due to expensive nature of `SELECT COUNT (*)` queries the system automatically detects long queries and if detected it will automatically omit `total` field in favour of `hasPrev` and `hasNext` fields.

Items

List of `items` .